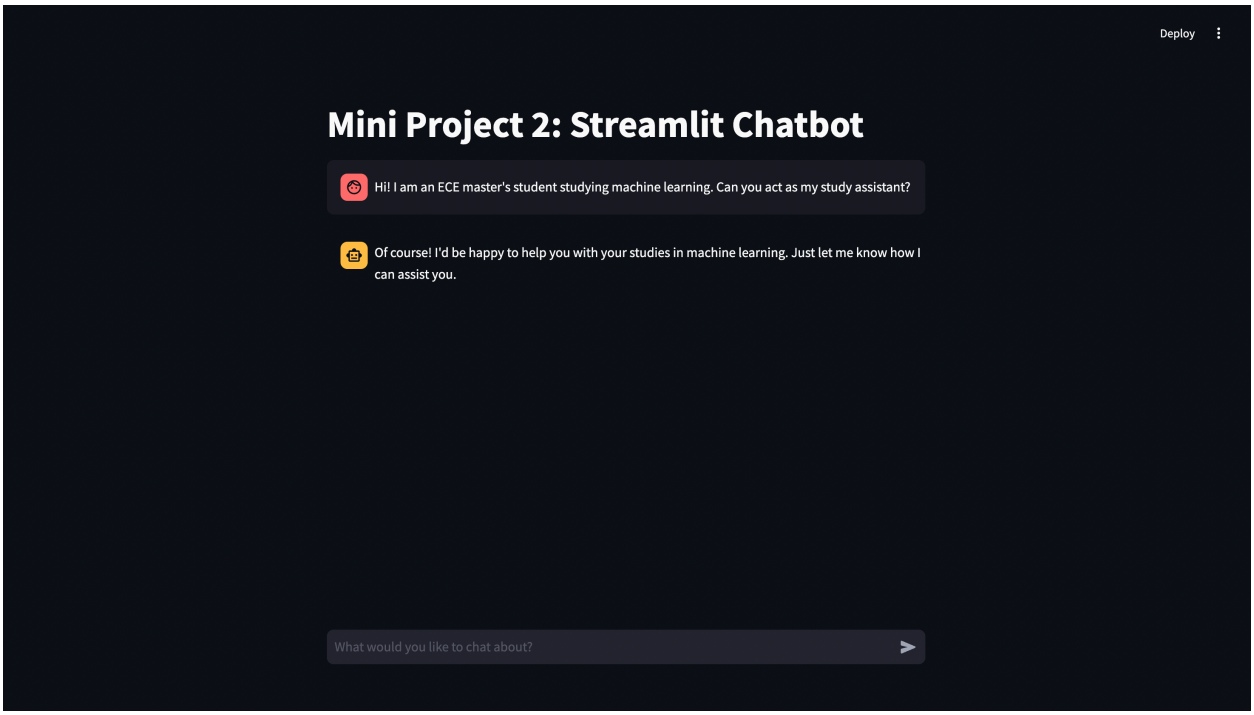
Mini Project 2 - Part 2: AI Chatbot Report

Test Case 1: Basic Greeting & Role Play

Purpose: To verify that the Streamlit interface accepts user input and the OpenAI API successfully returns a generated response.

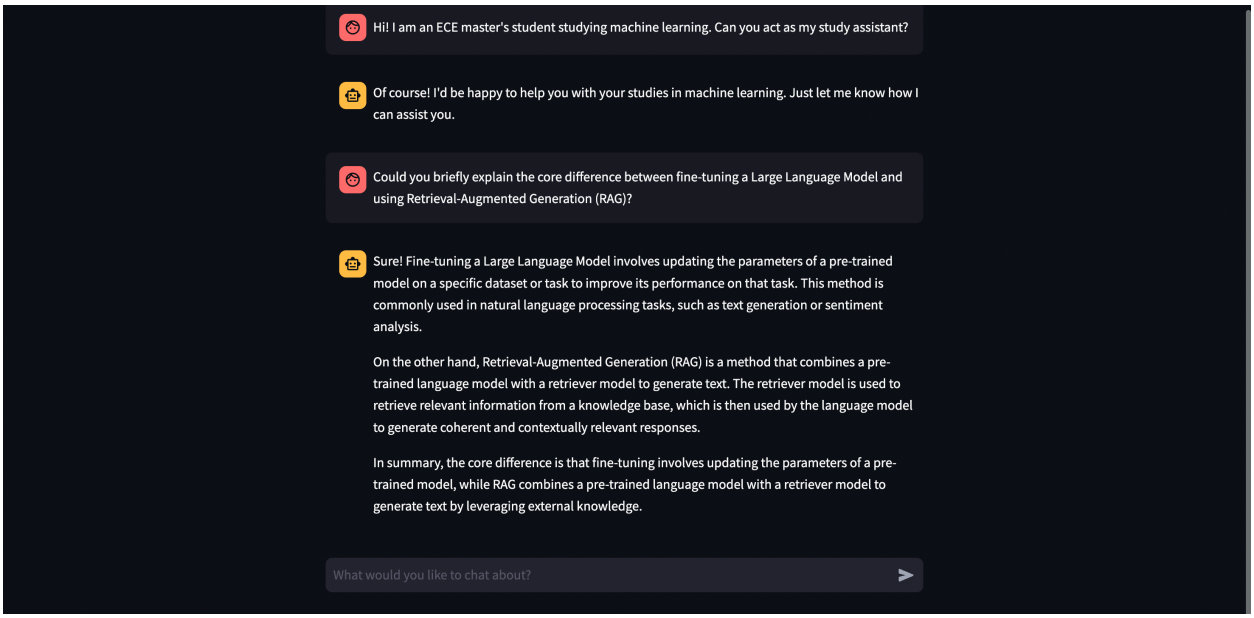
- **User Input:** Hi! I am an ECE master's student studying machine learning. Can you act as my study assistant?
- **Expected AI Response:** The AI should greet back and accept the role of a study assistant.



Test Case 2: Domain-Specific Query

Purpose: To test the model's ability to answer technical questions clearly within the chat interface.

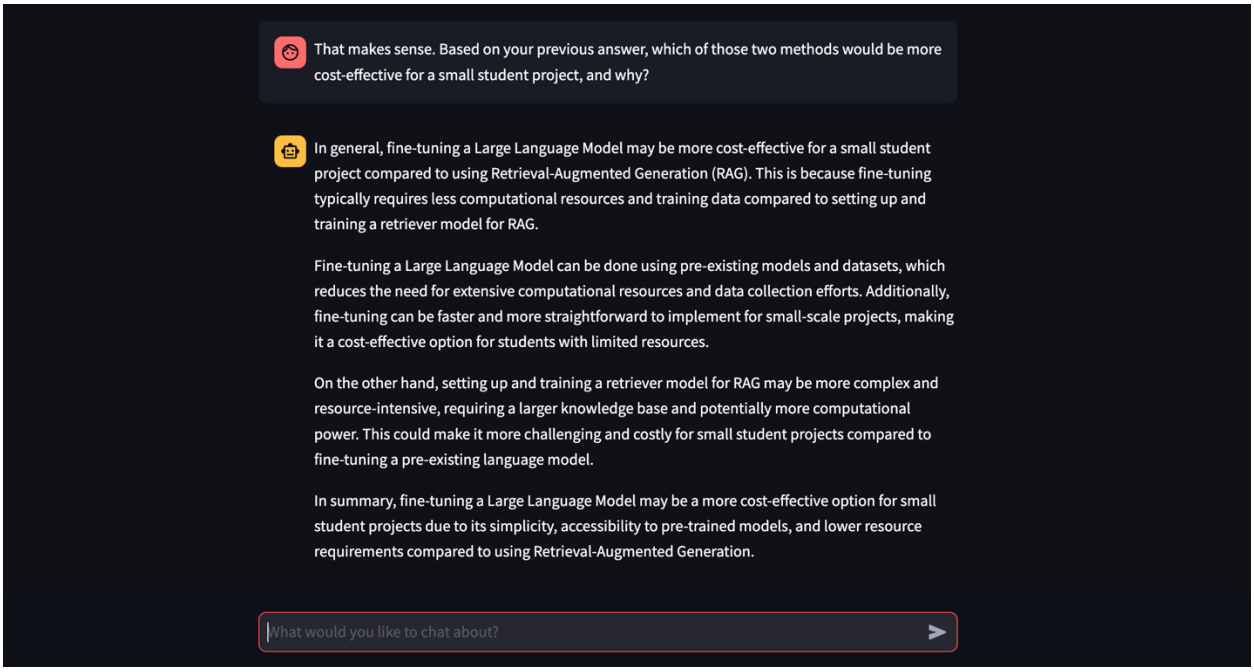
- **User Input:** Could you briefly explain the core difference between fine-tuning a Large Language Model and using Retrieval-Augmented Generation (RAG)?
- **Expected AI Response:** The AI should provide a technical comparison between fine-tuning (updating model weights) and RAG (fetching external context).



Test Case 3: Conversation History (Session State) Validation

Purpose: To demonstrate that the chatbot correctly utilizes st.session_state to remember the context of the current conversation.

- **User Input:** That makes sense. Based on your previous answer, which of those two methods would be more cost-effective for a small student project, and why?
- **Expected AI Response:** The AI should implicitly know that "those two methods" refers to Fine-tuning and RAG from the previous turn, and recommend RAG as the more cost-effective approach.



Test Case 4: Code Generation Task

Purpose: To test the bot's formatting capabilities in Streamlit.

- **User Input:** Can you write a simple Python function to calculate the cosine similarity between two word embeddings? Please use numpy.
- **Expected AI Response:** The AI should output formatted Python code using numpy.dot and numpy.linalg.norm.

Can you write a simple Python function to calculate the cosine similarity between two word embeddings? Please use numpy.

Sure! Here's a simple Python function that calculates the cosine similarity between two word embeddings using numpy:

```python
import numpy as np

def cosine_similarity(embedding1, embedding2):
    dot_product = np.dot(embedding1, embedding2)
    norm_embedding1 = np.linalg.norm(embedding1)
    norm_embedding2 = np.linalg.norm(embedding2)
    similarity = dot_product / (norm_embedding1 * norm_embedding2)
    return similarity

# Example usage:
embedding1 = np.array([1, 2, 3])
embedding2 = np.array([4, 5, 6])
similarity_score = cosine_similarity(embedding1, embedding2)
print(f"Cosine similarity between the two embeddings: {similarity_score}")
```

In this function, `embedding1` and `embedding2` are the input word embeddings as numpy arrays. The function calculates the dot product of the two embeddings and divides it by the product of their L2 norms to compute the cosine similarity.

- **User Input:** That's a great explanation. Could you summarize the pros and cons of RAG versus Fine-tuning in a comparative table?
- **Expected AI Response:** The AI should generate a well-formatted Markdown table comparing the two approaches, and render it as a visual UI table.

That's a great explanation. Could you summarize the pros and cons of RAG versus Fine-tuning in a comparative table?

Sure! Here is a summary of the pros and cons of Retrieval-Augmented Generation (RAG) versus Fine-tuning in a comparative table:

Aspect	Retrieval-Augmented Generation (RAG)	Fine-tuning a Large Language Model
Pros	- Incorporates external knowledge from a knowledge base	- Can be more fine-tuned for specific tasks
	- Can generate more contextually relevant responses	- Can achieve state-of-the-art performance in a specific task
	- Can improve model performance by leveraging external information	- Faster and more straightforward to implement for small-scale projects
	- Reduces the need for additional training data	- Can make use of pre-existing models and datasets
Cons	- Requires a separate retriever model and knowledge base setup	- May require more computational resources and training data
	- Complexity and resource-intensive compared to fine-tuning	- Limited to the capabilities of the pre-trained model
	- Retrieval latency for accessing external knowledge	

Please note that the suitability of RAG versus Fine-tuning may vary depending on the specific task requirements, available resources, and project goals.